SI 206 Final Project

Ethan Leach, Ethan Thompson, Saicharan Vemuri

https://github.com/ethantho/SI-206-Final-Project

Changes Since Our Grading Session

We fixed our stock prices graph to have readable labels for the X-axis that don't overlap.

Planned Project Goals

Our original goal was to collect data from three APIs: IGDB, OMDB, and AlphaVantage. These APIs would give us data on video games, movies, and the stock market respectively. From IGDB we planned to collect video game titles, developers, and release dates. From OMDB we would similarly collect movie titles, studios, and release dates. From AlphaVantage, we would collect stock prices at various different dates. We planned to then look for correlations between the frequency of video game/movie releases and stock prices. We then planned to create visualizations of our data and analysis using Matplotlib.

Actual Project Goals

From IGDB, we realized that the developer of a game was not useful information for the analysis we wanted to do, so we ended up only collecting game titles and release dates.

For movie collection, the goal was to collect a list of movies and their genre, along with their release date. We'd then compare the release dates to see what it was

For stocks data, our goal was to calculate average stock prices over the span of more than 20 years and compare the trends with movie and game releases.

Problems Faced

When collecting data from IGDB, we realized that we wanted to collect a random sample from all of the video games in the database, but the API did not have a random order option. To try to get close to a random sample, we started querying the API for titles starting with randomly generated two-character strings, getting us fairly random games from the database.

For the OMDb portion, the issue was roughly the same - there was no easy way to gather a random selection of games. To address this, we used a second API which was able to search for a list of movies with IMDb IDs, which we then used to search on OMDb.

When collecting stocks data, we had initially wanted to make comparisons using prices from the Dow Jones Industrial Average (DJIA), a prominent stock index often used to judge the state of the overall market. Since the DJIA holds accounts of a vast amount of companies' shares, using its data would allow us to make comparisons relative to a holistic view of the commerce market. However, we found that AlphaVantage did not provide stock market data for the DJIA, or for other large stock indices such as the S&P 500, and since we were to use freely available APIs, we were not able to find any way to get that data in its original form. To remedy this, we decided to use the DJIA's ETF Trust, which is a representation of DJIA shares that matches its prices at a proportional value. Since this information *was* hosted on AlphaVantage, we were able to derive comparisons using this data.

When combining the tables together, we attempted to use a JOIN to join the tables by year and count the values. However, this did not work, so to address this, we had to look into making subquery expressions, which worked well.

Calculations

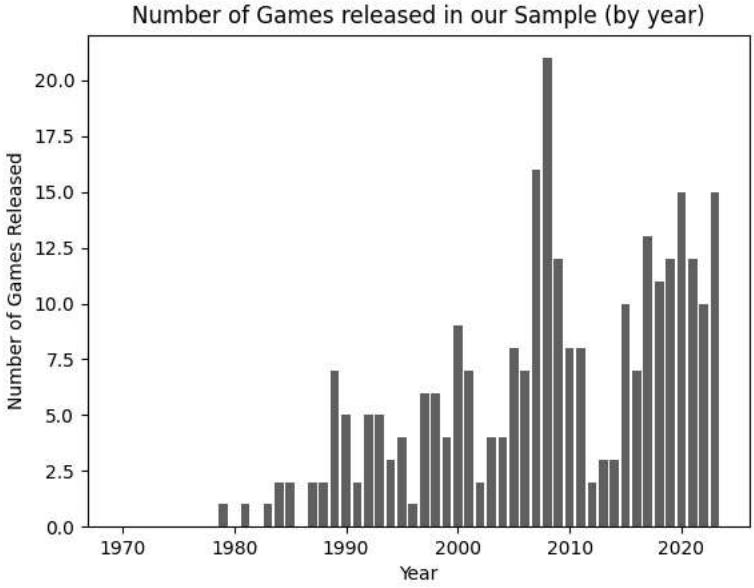
For games, we counted the number of games in our sample that were from each year, and used this to create a bar chart.

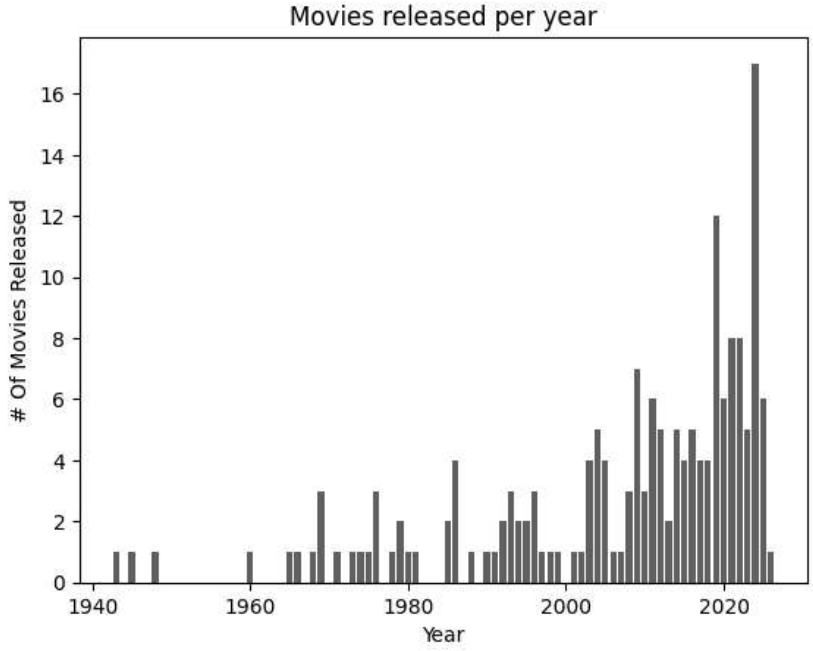
For the movies, we group up the movie counts by genre with an SQL query, then create a stacked bar chart (as can be seen in the visualizations below).

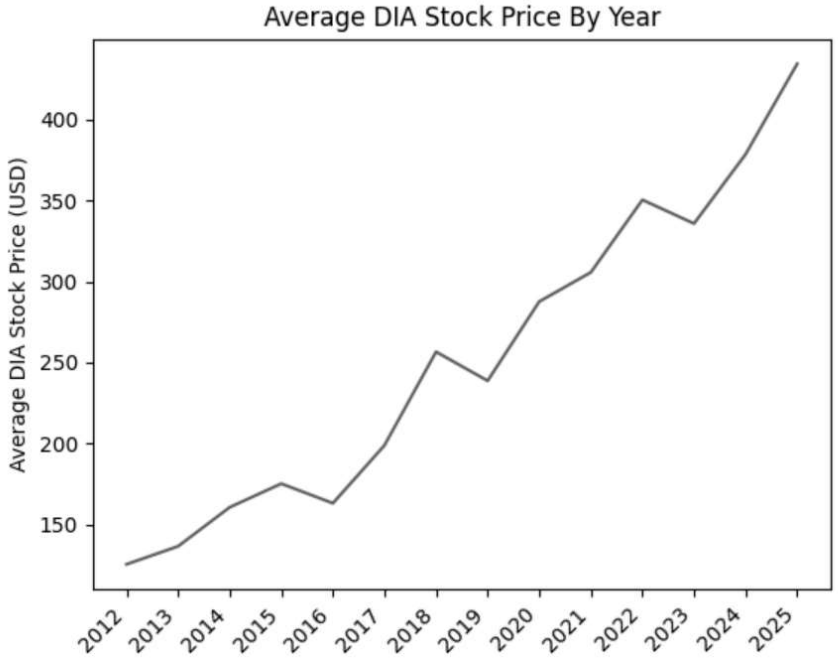
For stocks data, we gather monthly prices for market highs and lows, combine them to produce a monthly average, and display them in a line chart.

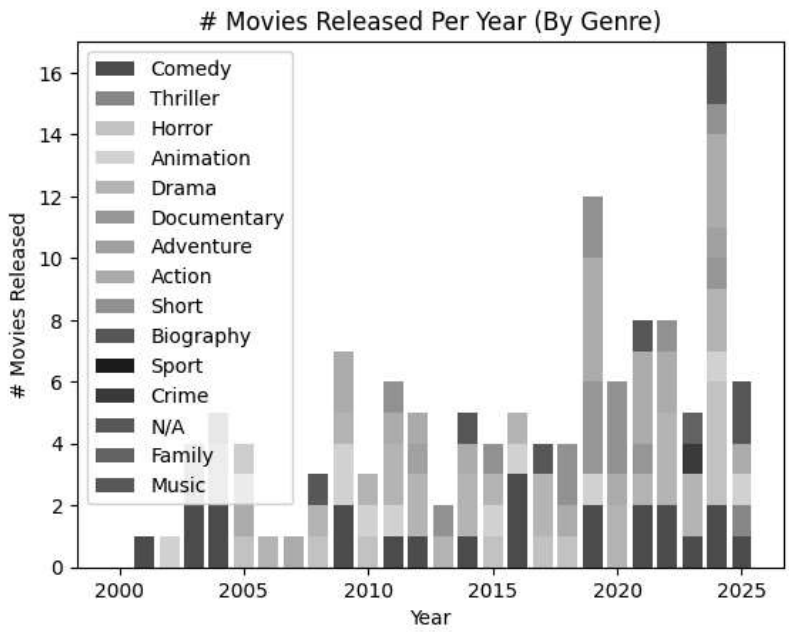
Then, to mix the data, we simply run an SQL query which runs subqueries per year to group together our data by year, then display it with a grouped side-by-side bar chart along with a line chart.

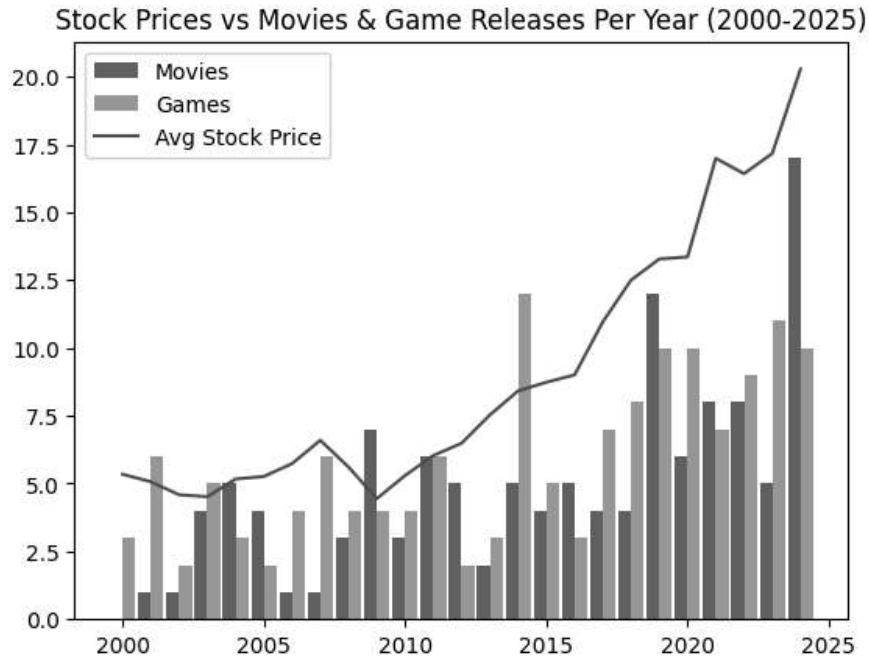
Visualizations


Number of Games released in our Sample (by year)
Number of Games Released
Year
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
1970
1980
1990
2000
2010
2020


Movies released per year
Of Movies Released
Year
0
2
4
6
8
10
12
14
16
1940
1960
1980
2000
2020


Average DIA Stock Price By Year
Average DIA Stock Price (USD)
400
350
300
250
200
150
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025





Instructions for Running Our Code

To populate the database with data from IGDB, OMDB, or AlphaVantage, simply run

python3 <Name_of_file.py>

where <Name_of_file.py> is game_data.py, movie_data.py, or stock_market.py, in the terminal several times until the desired amount of data is collected. Each run will add up to 25 entries. A shorthand for adding data is just:
python main_data.py

To generate visualizations, you just need to run:

- python run_visualizations.py

Function Documentation

Lorem ipsum dolor sit amet

- movie_data.py
  - initialize_tables(conn: *Connection*) → *None*
    - Creates the tables in the database relevant to the movies.
  - get_random_movie_info(conn: *Connection*, cycles: *int*) → *None*
    - Fetches random basic data from IMDb, and adds it to the database to be later used.
  - get_genre(genre: *str*, conn: *Connection*) → *int*
    - Fetches the genre id associated with the genre name from the database.
  - parse_movie_obj(imdb_id: *str*, data: *dict*, conn: *Connection*) → *None*
    - Converts the dictionary *data* to be of the form expected by the database, and inserts it.
  - fetch_omdb_info(conn: *Connection*) → *None*
    - Fetches a list of 25 IDs collected by the random movie info, and then queries OMDb to gather more information about them, to then insert into the database.
  - main(conn: *Connection*) → *None*
    - Initializes the tables (if necessary), gets a list of many random movie IDs, then fetches and adds them to the database.
- movie_visualize.py
  - get_index(array: *list[tuple]*, index: *int*) → *list[Unknown]*
    - Creates a list of values from a list of tuples, taking the value at the specified index.
  - gen_array(values: *list[tuple[int,str,int]]*) → *list[int]*
    - Converts a list of tuples into an array to be used for further calculations.
  - add_arrays(l1: *list[int]*, l2: *list[int]*) → *None*
    - Adds the values from L2 to L1, in place on L1.
  - main(conn: *Connection*) → *None*
    - Creates both visualizations and output csv files for both movies by year alone and movies by genre by year.
- combined_visualize.py
  - main(conn: *Connection*) → *None*

        - Creates a visualization of all the data, combined into one chart, as well as outputting the data into *combined.csv*.
- game_data.py
    - random_search() → *string*
        - Generates a random two-character ASCII lowercase string.
    - main(conn: *Connection)* → *None*
        - Adds up to 25 games with a random title to the database.
- game_visualizations.py
    - year_to_timestamp(year: int) → *int*
        - Converts year into the number of seconds from January 1st, 1970 to the start of that year.
    - main(conn: *Connection)* → *None*
        - Calculates the number of game releases in the database that occurred in each year. Outputs this data to game_calculations.txt and generates a bar graph of it to games.png
- stocks_data.py
    - main(conn: Connection) → *None*
        - Takes data from AlphaVantage for the DIA trust, 20 months at a time, and adds all details (open price, high price, low price, close price) to the database
- Stocks_visualize.py
    - main(conn: Connection) → *None*
        - Gets the high and low stock price of each month, sorted earliest to latest, calculates the average stock price of each month based on the high and low (uses arithmetic average), and outputs data into a matplotlib line graph and a text file stock_calculations.txt in a readable format

Resources Used

Date	Issue Description	Location of Resource	Result (did it solve the issue?)
4/22/2025	The labels on the stocks graph can't be read	https://stackoverflow.com/questions/10998621/rotate-axis-tick-labels	Yes, now we only write the year, and these years are rotated so that they don't overlap.